

Microsoft Project: AI-Expense Sharing App (Project 4)

Participants: Sofia Orieshkina, Thomas Cormican, Joel Timothy Csadi, Mariia Korzachenko, Sean Rory Mullen

Work Credits:

Thomas Cormican: backend and database, prototype, report

Sofia Orieshkina: frontend, prototype, report

Joel Timothy Csadi: UML and database, AI, report

Mariia Korzachenko: logo, name, prototype, UML, report

Sean Rory Mullen: prototype, feedbacks, presentation

Part 1: User stories and requirements

1. Scanner and AI receipt processing

As a user, I want to be able to take a photo of a receipt and have the app automatically extract the items and total so that I do not have to enter the bill manually. This feature is triggered from the main screen when a user selects the option to add a new event. The user is prompted to take a photo of the receipt using their device camera. The image is sent as a POST request to the backend where it is processed by an AI Azure document intelligence model. The model extracts structured data from the receipt including individual item descriptions, item prices, tax, tip, and the overall total. The parsed receipt data is presented to the user for review on a screen. They can make corrections to any item name or price before proceeding to account for cases where the AI misreads a value. Once the user confirms the receipt contents, the data is saved to the database under the relevant event, and the receipt image is stored in cloud BLOB storage.

2. Receipt Manual Input

As a user, I want to be able to input the receipt manually if I don't want to use my camera right now, so that I am using Manual Input of the receipt. Manual Input requires users to put in the items and some information about them: their names, prices and costs to the manual input frame. Then program will sum all of them up and proceed with it to the splitting part. Once the user chooses how they want to split the receipt, it goes to reduction algorithm to simplify the debt, ensuring the minimum number of transactions are needed to pay everyone back.

3. Expense Sharing by items

As a user, I want to be able to assign individual items from a receipt to specific people so that each person only pays for exactly what they ordered. This feature becomes available after a receipt has been scanned or entered manually and the user has selected the people taking part in the event. On item screen, user can tap on any item and choose which person or people it should be assigned to. If multiple people share an item, the cost of that item is divided equally among them. Once all items have been assigned, the app calculates each person's total by summing the costs. A debt record is created for each person who owes money to the person who paid the bill upfront.

4. Division by percentage or equal division

As a user, I want to be able to select to pay for a receipt by dividing the costs evenly by equal proportion or by specific percentages so that we can pay based on a fraction of the price.

This appears after scanning a receipt via Azure AI Document Intelligence receipt scanner model or via manually input of the receipt. Once the receipt is agreed upon, we select who will be paying for the bill via friends or groups (feature). Once this is finalized, we're able to select how we pay the bill. If the bill is split equally, the total cost of the bill will be divided by the number of people in the receipt event. If the bill is split by percentage per person, people can pay a specific proportion of the bill from a percentage from (1-100%). Total percentage among the group must sum up to 100% and the costs must be equal to the final bill. Once this is finished, the price for each user is resolved and upon clicking done, the receipt history will be updated and there will be debt required to be given to the payer of the bill.

5. Single User Mode

As a user, I want to be able to pay for a receipt alone so that I can record my personal finances from receipt history. The user scans a receipt or uses manual entry. The items and the cost are visualized and the user proceeds to "add people and groups" to the bill screen. Since the user only wants to include themselves, they do not add any more people to the bill event and press continue. Since the user is paying the bill by themselves, the cost of the bill screen is displayed. There's no need for the payment method selection screen to appear since the bill is paid in full alone by the user. Once the user presses done on the bill screen where the items and total cost is visualized, the receipt history of the user is updated, and no debt is allocated since the user paid it themselves. The user can press on a receipt event and view the past receipt contents (items, payment date) and costs for a record of their personal expenses.

6. Creation of the Group

As a user, I want to create a group with people which I can use without adding the same people all the time to the receipt, so that I am using the Create Group feature. For pre-create groups, users should go to the Friends part of the app and add the new Group, where users can add all the users with whom they are splitting the receipt usually. For actual splitting user should input the receipt, manually, using OCR or importing it, then check the data of it and in users adding they can choose the group to add, such that on the screen will appear all the usernames of the Group. Then it will be the same procedure as with the input users.

7. Optimization and minimalization of owes

As a user, I want the app to simplify the debts between people in a group so that the number of payments needed to settle everything is as small as possible. After one or more events have been completed within a group, it is common for debts to exist in multiple directions between members. For example, person A may owe person B from one event, while person B owes person A from a different event, and person C owes both A and B from a third event. Without optimization, each of these debts would need to be settled individually, leading to unnecessary back and forth payments. The optimization feature calculates the net balance for each person across all unsettled debts in the group. The simplified debt view is displayed on the friend's screen, showing each person what they owe and to whom after optimization. This feature reduces the amount of payments needed to pay the debt.

8. Ability to show the paid debt

As a user, I want to show that my debt is paid, so that I am using the “debt paid” bottom in the friend’s part. In the Friends section of the app users can see the sum they owe to person to how much user owes them. By simply clicking on the amount, they can set how much they paid to the person. It will be automatically applied to the person’s database and to the database of their friend.

9. History of spending (Group and single)

As a user, I want to be able to see the receipts history, so that I can go to the History section of the app. In the history user can see dated receipts that user did previously; by clicking on it, app is showing the items, their names, prices and quantity being used in the previous splitting. In the bottom it is the bottom ‘next’ by proceeding with it, user will see the actual split and the participants of the receipt. As the single-user history receipt it will be the sum up of the items in previous page. This data can be edited if some of the items were wrong. If it is changed, all the users will see the changes and their debt will be automatized and changed in the Friends Section.

10. Self-hosting App

As a user, I want to be able to Self-Host an app, so that I am in complete control of my data, models, and infrastructure. When a user decides to set up the system, they utilize Docker to deploy the entire stack, including the database, frontend, and backend to their own private infrastructure. Most of the system’s components run locally due to the boxed structure of the Docker. The customer ensures that all receipt processing, cost computation, and AI inference take place within their own environment by avoiding required cloud dependencies. Transparency and long-term ownership are provided to the user by this setup, which serves as a private, flexible financial assistant with no outside parties having access to their private data.

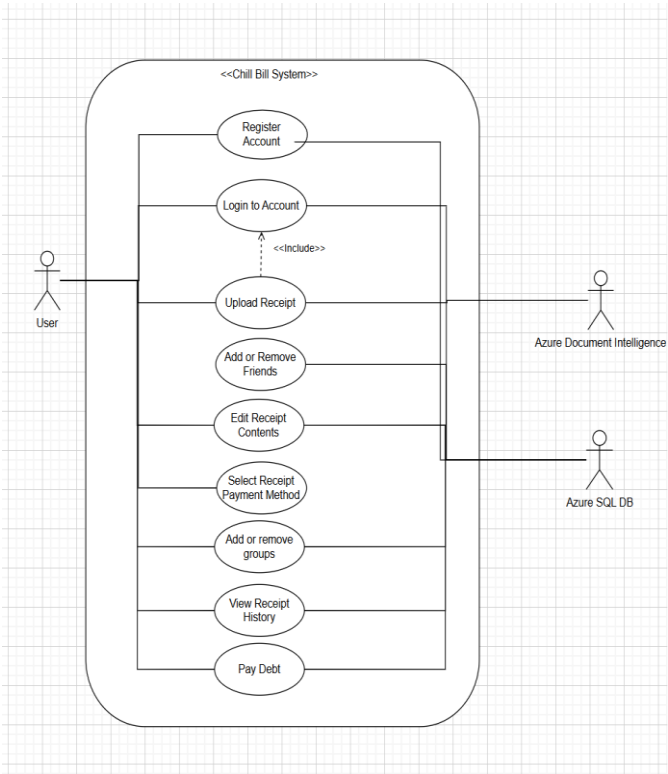
Part 2: UML diagrams for the software system

1. Use Case diagram

In this system, there are three actors, one of them is the user (human actor) and two of them are external actors or systems that communicate with the system and are responsible for data persistence and communicating with a hosted AI model via serverless Azure functions.

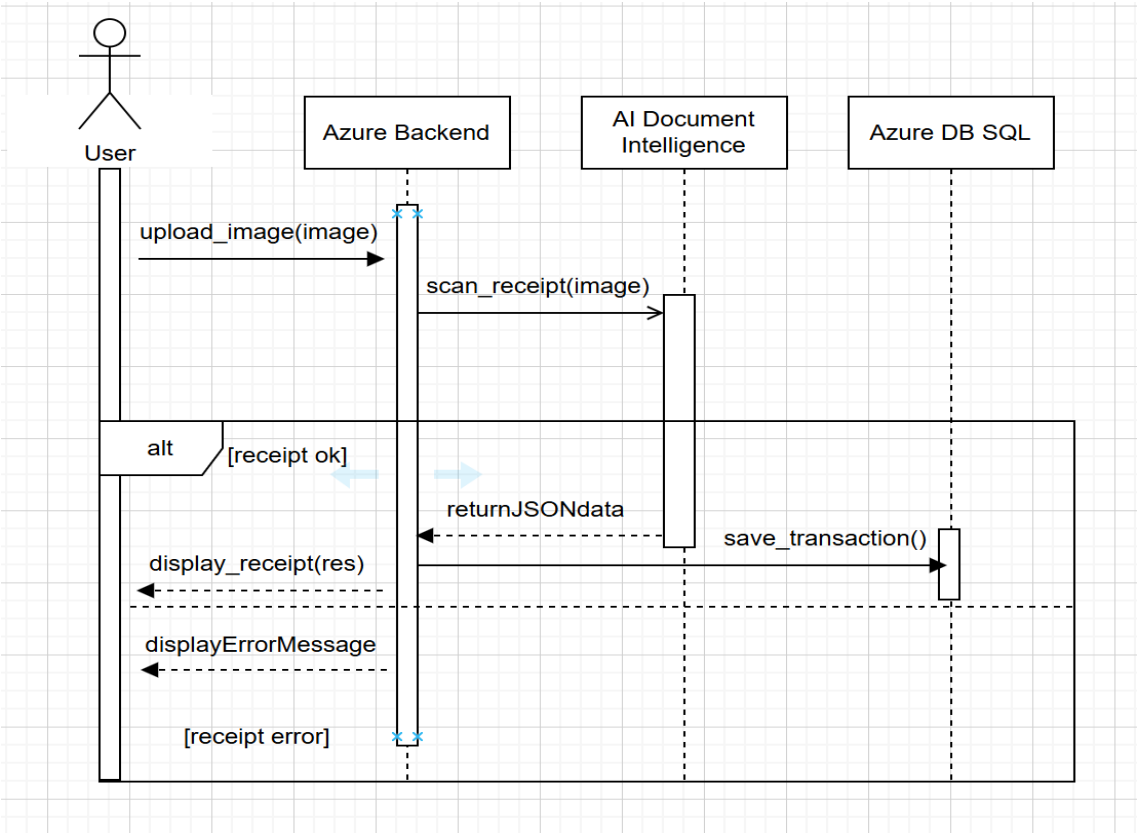
The User can perform activities such as registering to an account, login, upload a receipt, add and remove friends and groups, edit a receipt’s contents, select a receipt payment method, view receipt history and pay debt to others. To access many of these features, the user must be logged into an account, so it assumes that when a user uploads a receipt or adds a friend, an account is created and is logged into by the user.

The external actors interact with these use cases by storing data in an SQL database that relates to a particular entity such as users, receipts etc. whenever an action is performed by a user relating to that. When uploading a receipt, it interacts with the Document Intelligence model on Azure and sends a request and receives a response whether the receipt was successful or could not fulfill the request as desired.



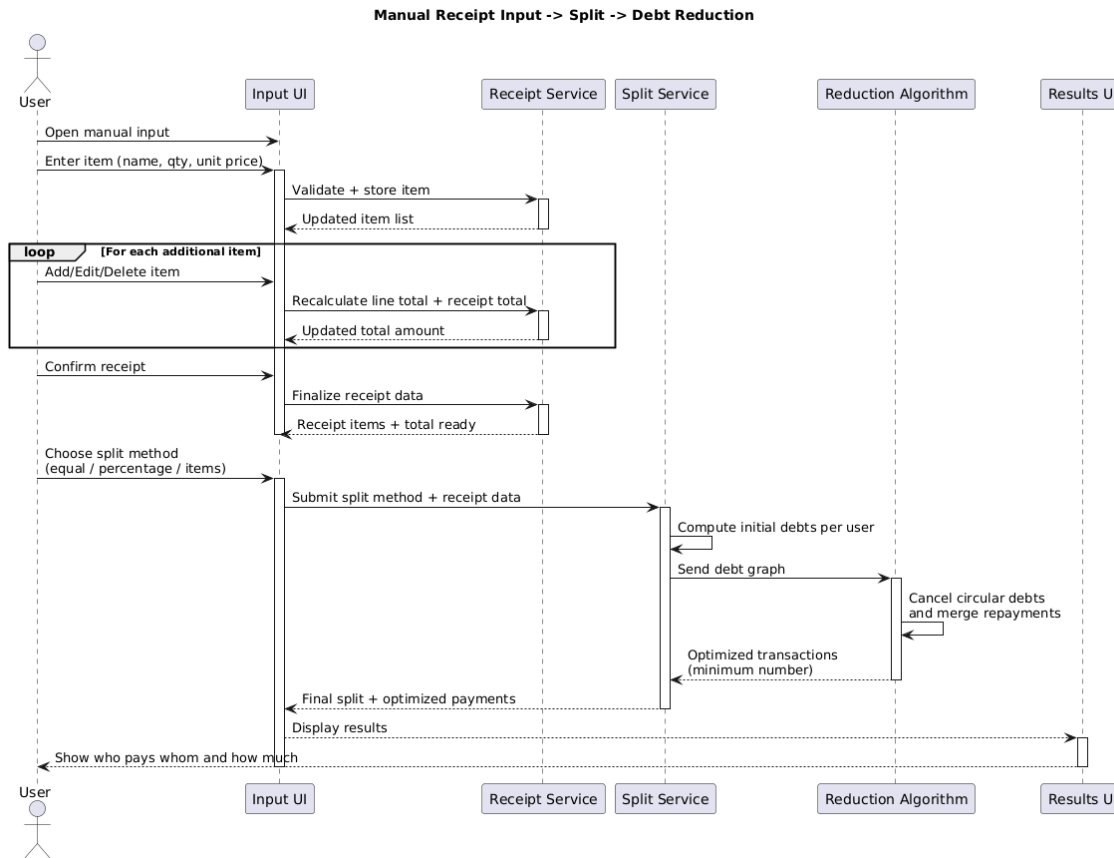
2. Sequence diagram

(i) AI Receipt Scanner



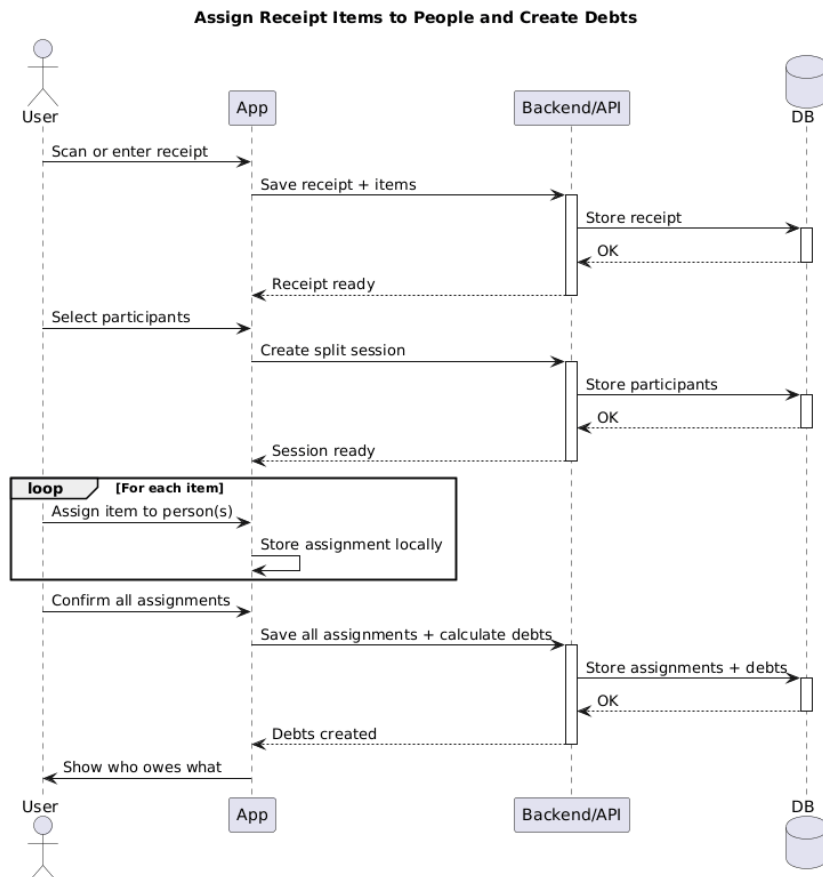
A user uploads an image(.png, .jpeg) of a receipt and the react sends a POST request to the Azure serverless backend route scan_receipt with the image in the body of the request. If the receipt is successfully extracted, it saves the entry to the database in the receipts table and the receipt image to the BLOB storage in Azure.

(ii) Receipt Manual Input



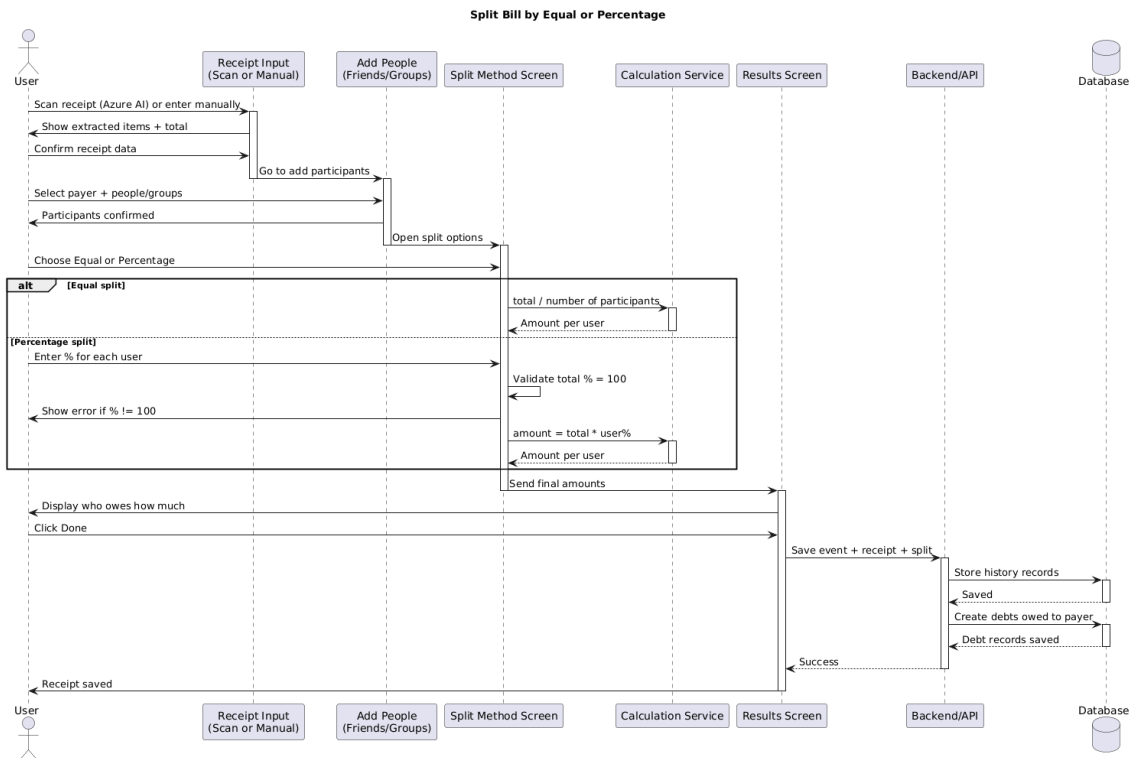
The process for manually entering a receipt, calculating its division among users, and maximizing the resulting debts is shown in this sequence diagram. Through validation, debt calculation, and the final "Reduction Algorithm" that condenses transactions into a minimum number of payments, it maintains the data from the first Interface input.

(iii) Expense Sharing by items



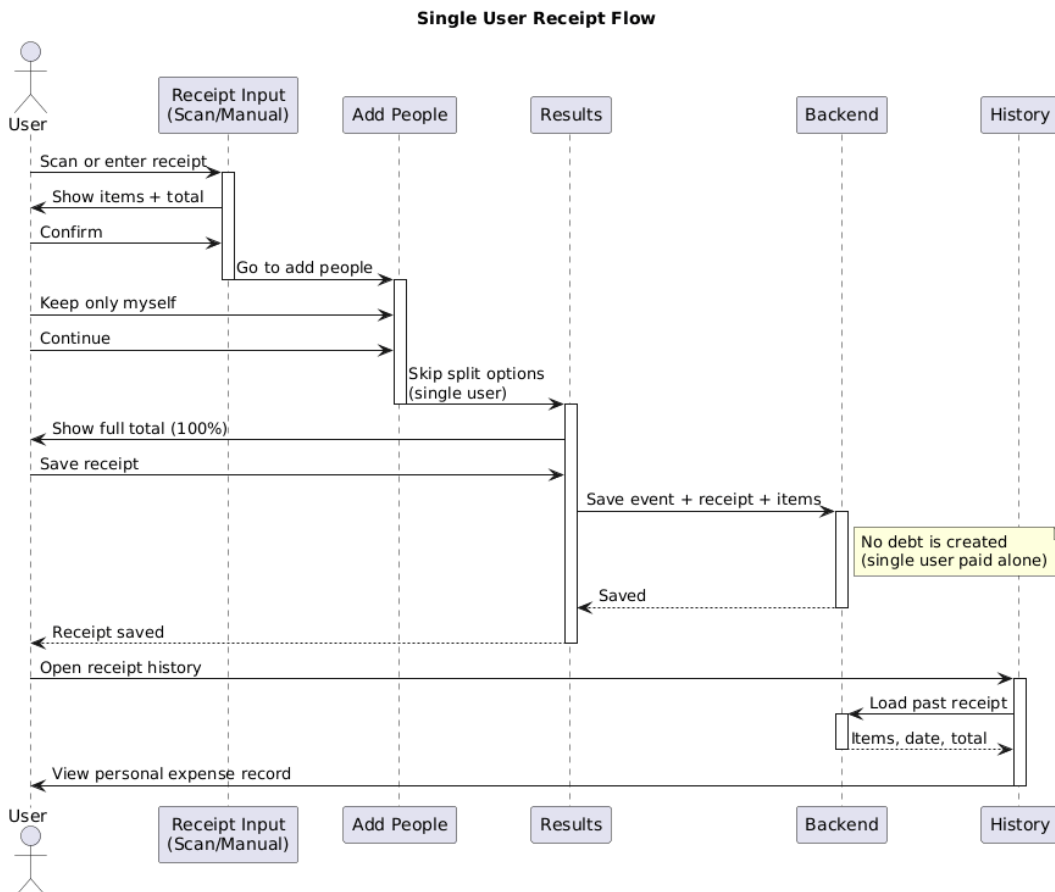
After a receipt is scanned or entered and participants are selected, users assign each receipt item to one or more people in the app. Shared items are split evenly, individual totals are calculated from assigned items, and debt records are created so each person owes their exact amount to the person who paid upfront.

(iv) Division by percentage or equal division



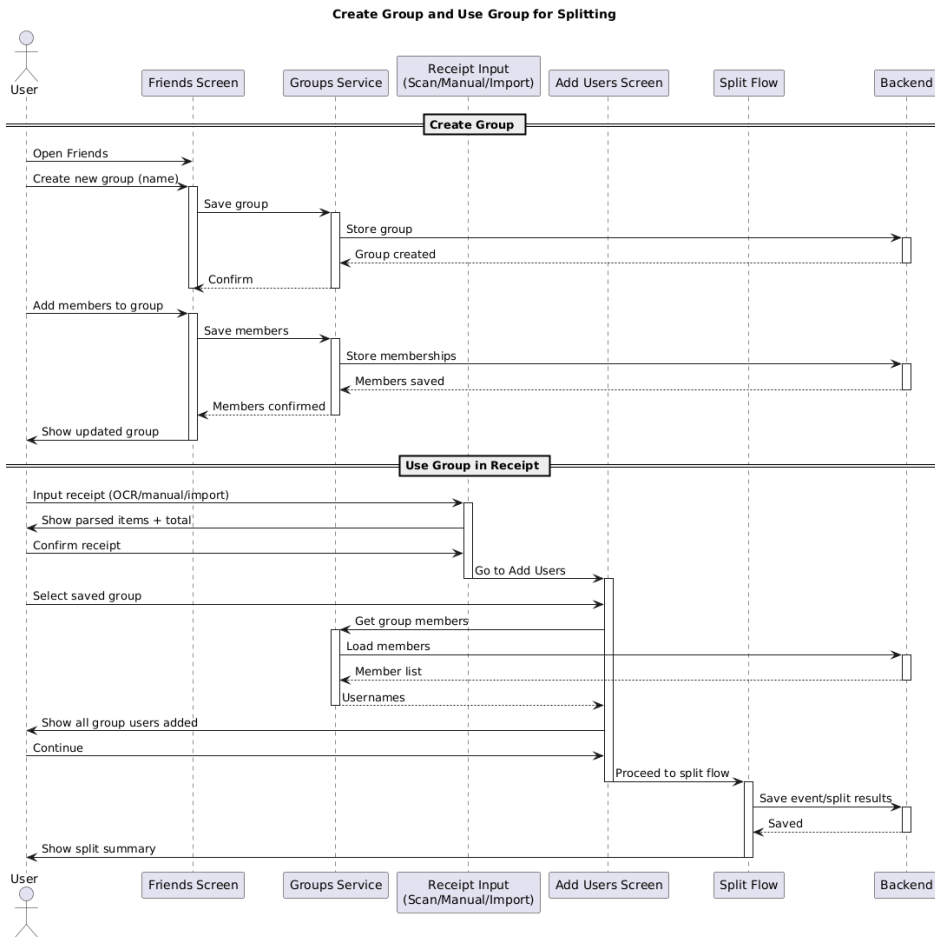
After a receipt is scanned or entered, this sequence diagram shows the user flow for splitting a bill using either an equal or percentage-based method. It follows the logic of participant selection, the calculating service, and the backends' database storage of the completed debt records.

(v) Single User Mode



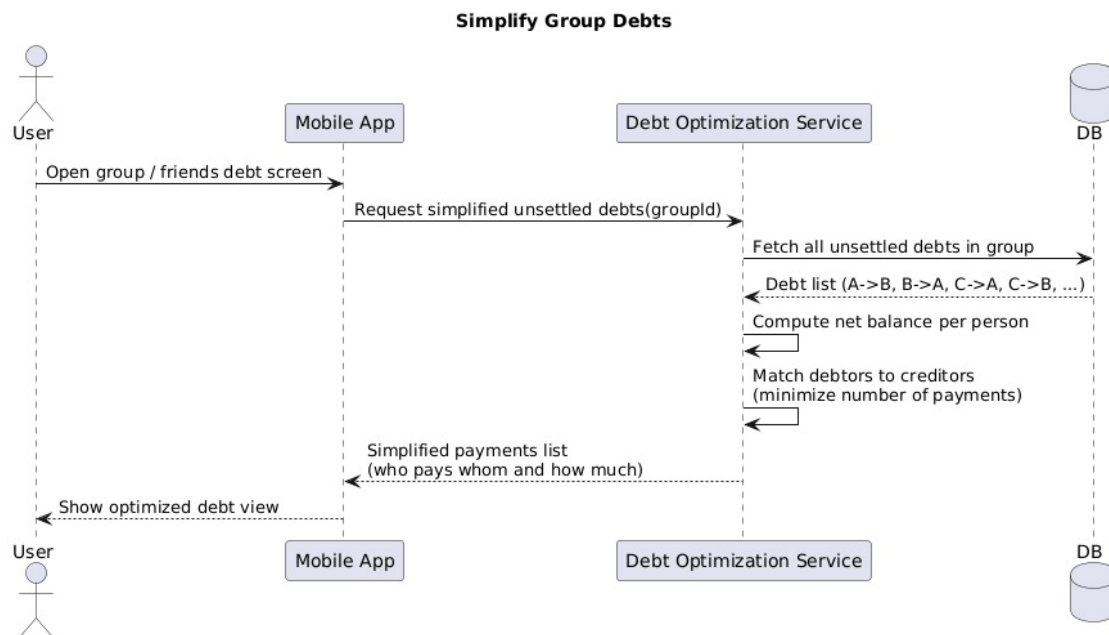
The process for a single user to register a personal expense without splitting it with others is shown in this sequence diagram. By instantly saving the receipt to the backend and making it accessible for later review in the user's history, it demonstrates how the system may avoid split logic and debt buildup.

(vi) Creation of the Group



The two steps of forming a social group and then using it to split a receipt are described in this sequence diagram. Using the Groups Service, which stores the data in the backend, the user creates a group name and adds members in the first phase. To automatically fill the participant list for a split, the second phase demonstrates how a user can easily select this pre-saved group during receipt entry. The split summary is then processed by the system, which then records the transaction event to the database.

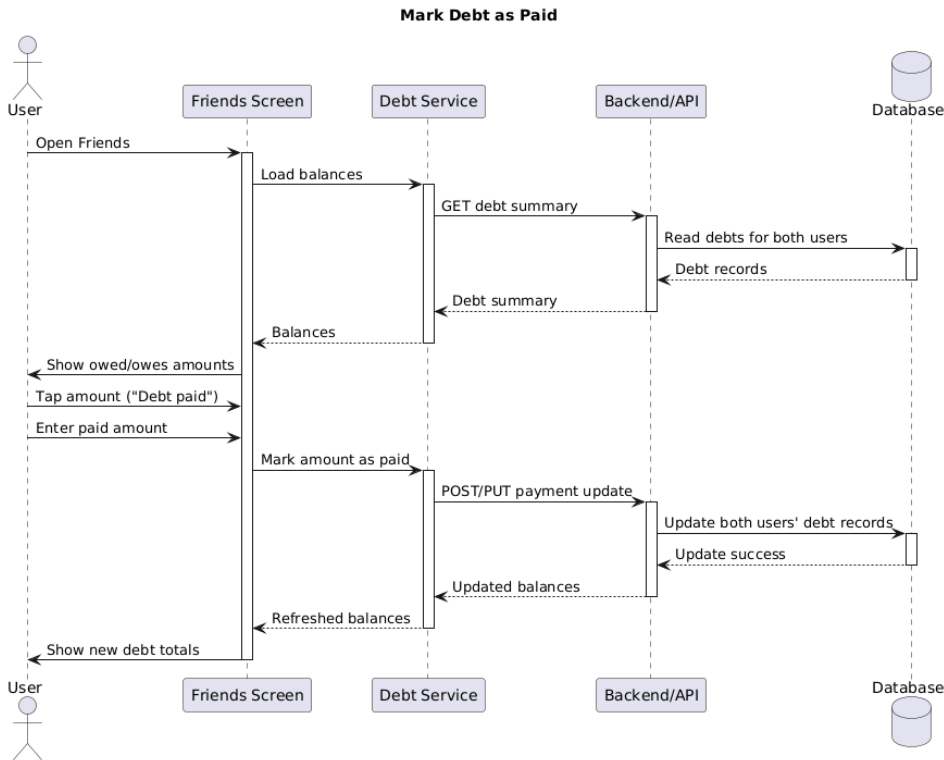
(vii) Optimization and minimalization of owes



When a user opens the group debt view, the app collects all unsettled debts for that group and calculates each member's net balance (total owed minus total to receive).

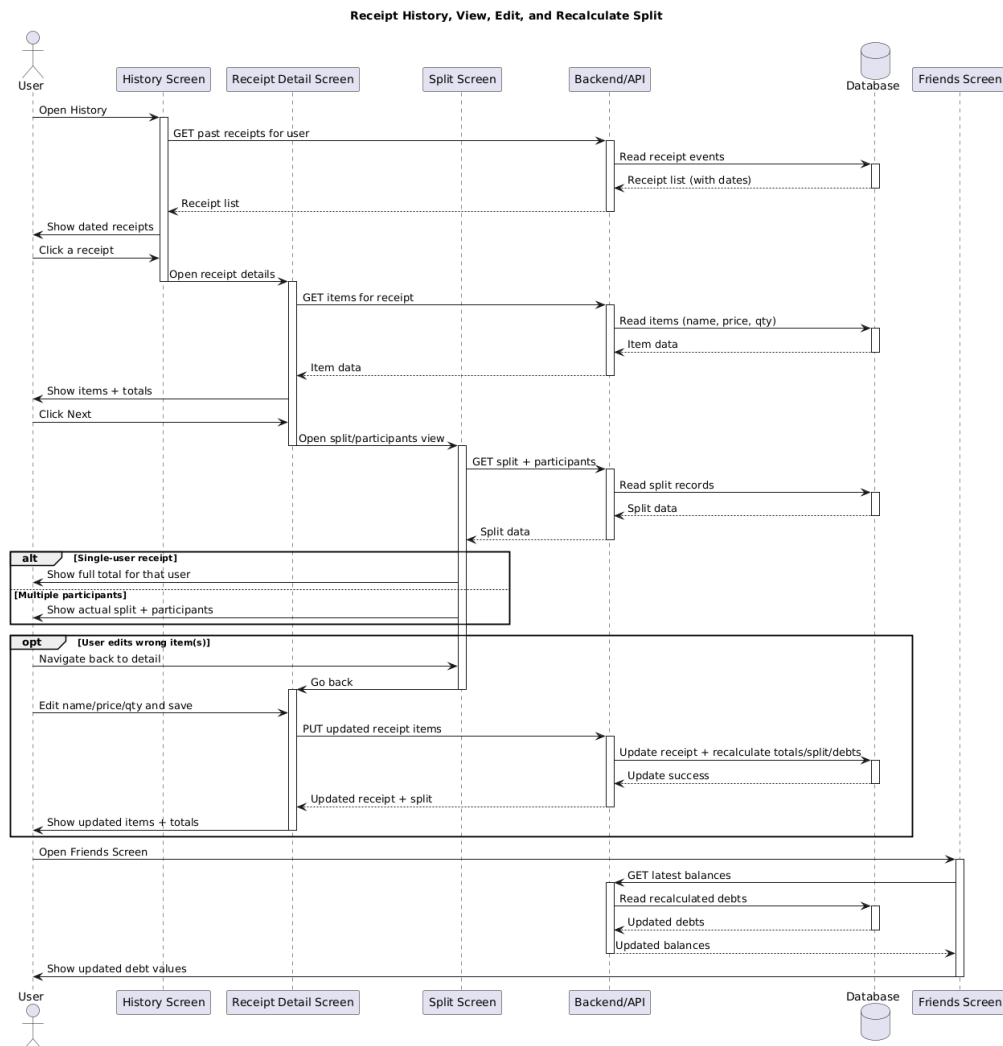
It then optimizes these balances into a smaller set of payments, so users see a simplified list of who should pay whom with fewer transactions overall.

(viii) Ability to show the paid debt



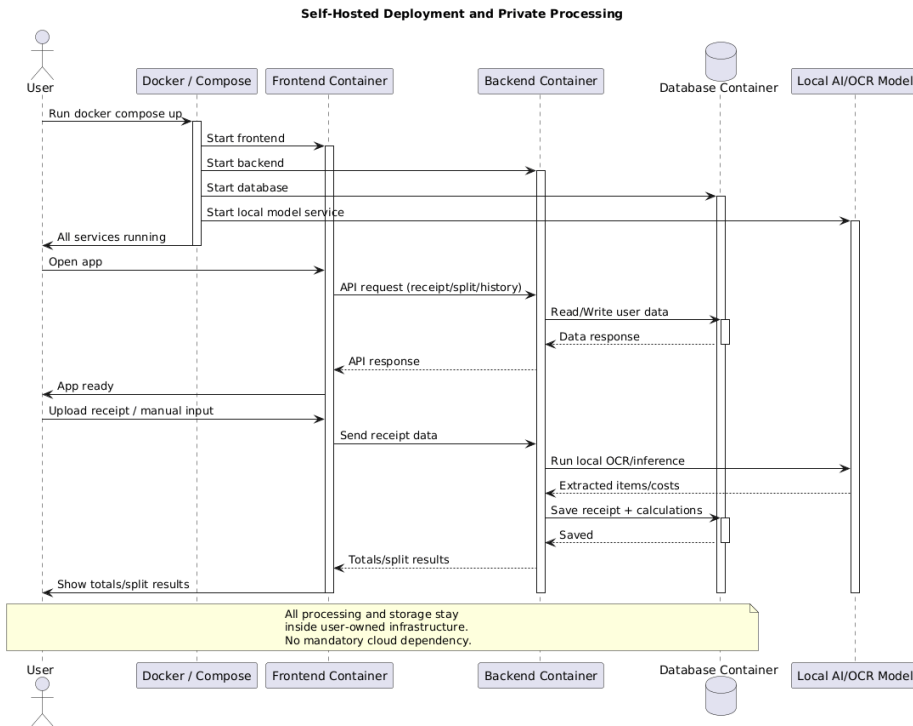
The technical process for settling an outstanding balance between friends within the program is depicted in this sequence diagram. A data fetch from the Debt Service and the backend database is initiated when the user views their current balances. The system synchronizes the records for the debtor and the creditor by performing a POST/PUT update after the user enters a payment amount. The user interface is refreshed at the end of the process to show the user the most recent debt totals.

(ix) History of spending (Group and single)



The lifecycle of managing previous transactions, from reviewing a history to modifying and updating previous splits, is shown in this sequence diagram. With conditional logic managing both single-user and multi-participant views, users can access detailed receipt and participant data from the database via the Backend/API. Users may change item data via an optional path, which causes the system to automatically recalculate totals and update debt amounts throughout the platform. In order to ensure data consistency, the flow ends by making sure the Friends Screen shows these updated financial values.

(x) Self-hosting App



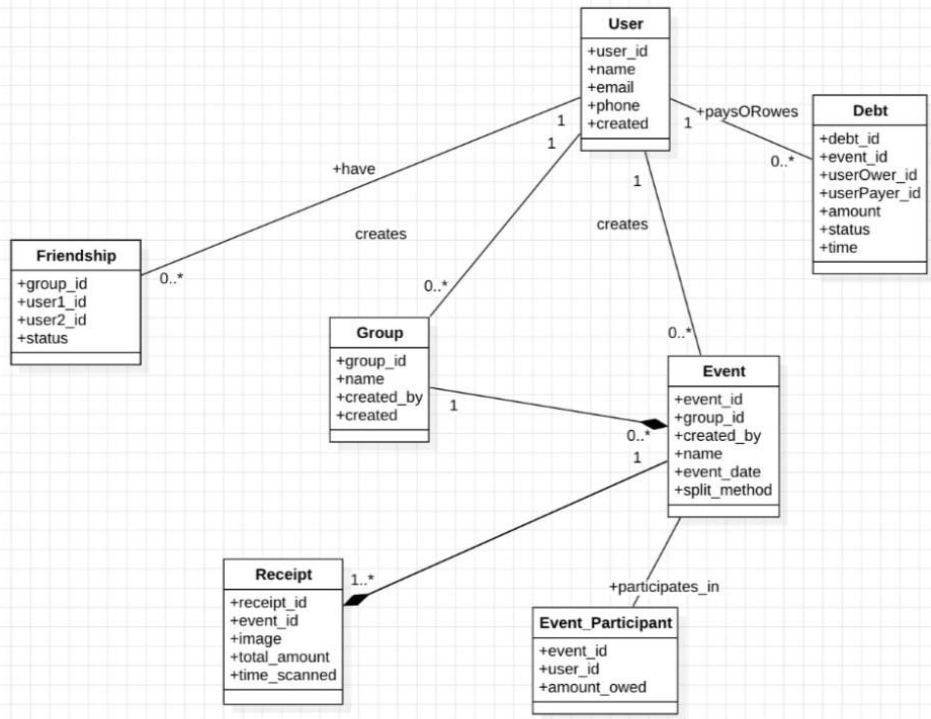
The lifecycle of managing previous transactions, from reviewing a history to modifying and updating previous splits, is shown in this sequence diagram. With conditional logic managing both single-user and multi-participant views, users can access detailed receipt and participant data from the database via the Backend/API. Users may change item data via an optional path, which causes the system to automatically recalculate totals and update debt amounts throughout the platform. In order to ensure data consistency, the flow ends by making sure the Friends Screen shows these updated financial values.

3. Class diagram

This report documents the Entity Relationship (ER) diagram for the Receipt Splitter application. The application allows users to photograph receipts, split bills between friends, track shared expenses across groups, and manage outstanding debts. The database has been designed to be lean and practical, focusing on the core features needed to deliver a working product. The design follows a simplified approach with 7 core entities, avoiding unnecessary complexity while preserving all the functionality required by the application.

The Receipt Splitter application supports the following key features:

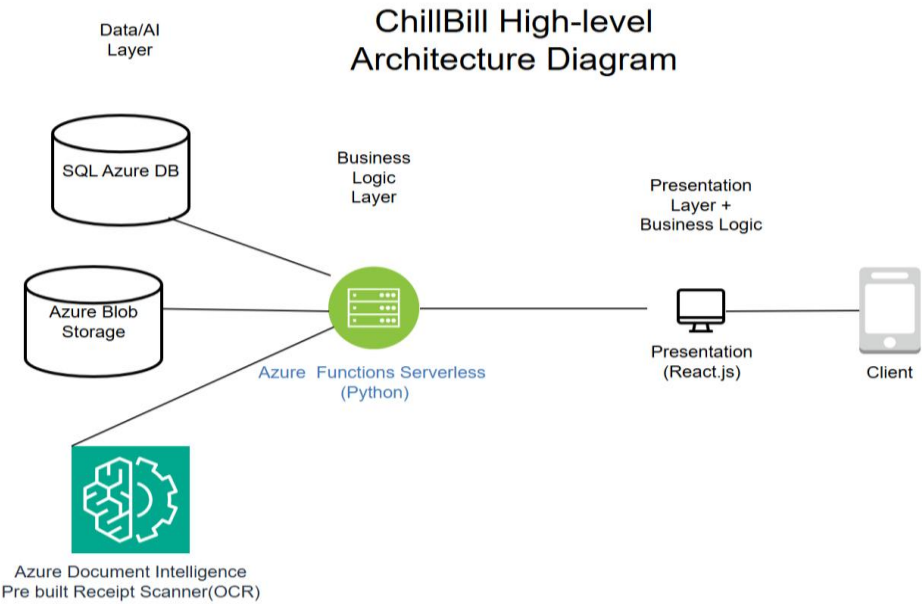
- Receipt scanning: Users photograph a receipt and the app extracts the total amount
- Bill splitting: The check can be divided equally, by percentage, by weight, or by item
- Groups: Users can create persistent groups that retain a history of shared events
- Friends: Users can add friends, pay on their behalf, and track repayments
- Debt tracking: The system calculates and monitors who owes what after each event



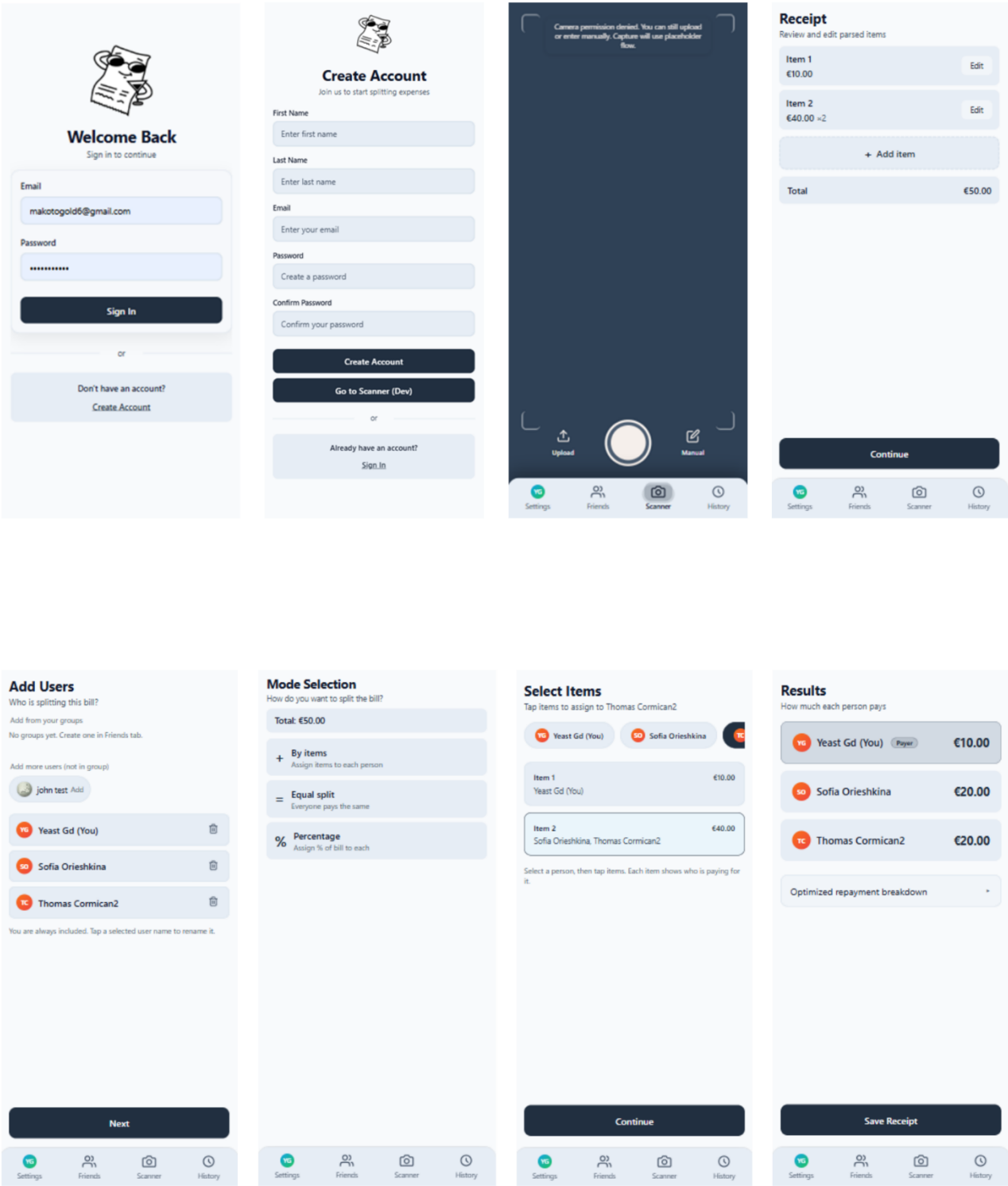
The database is composed of 7 entities. Each entity is described below with its attributes, data types, and purpose.

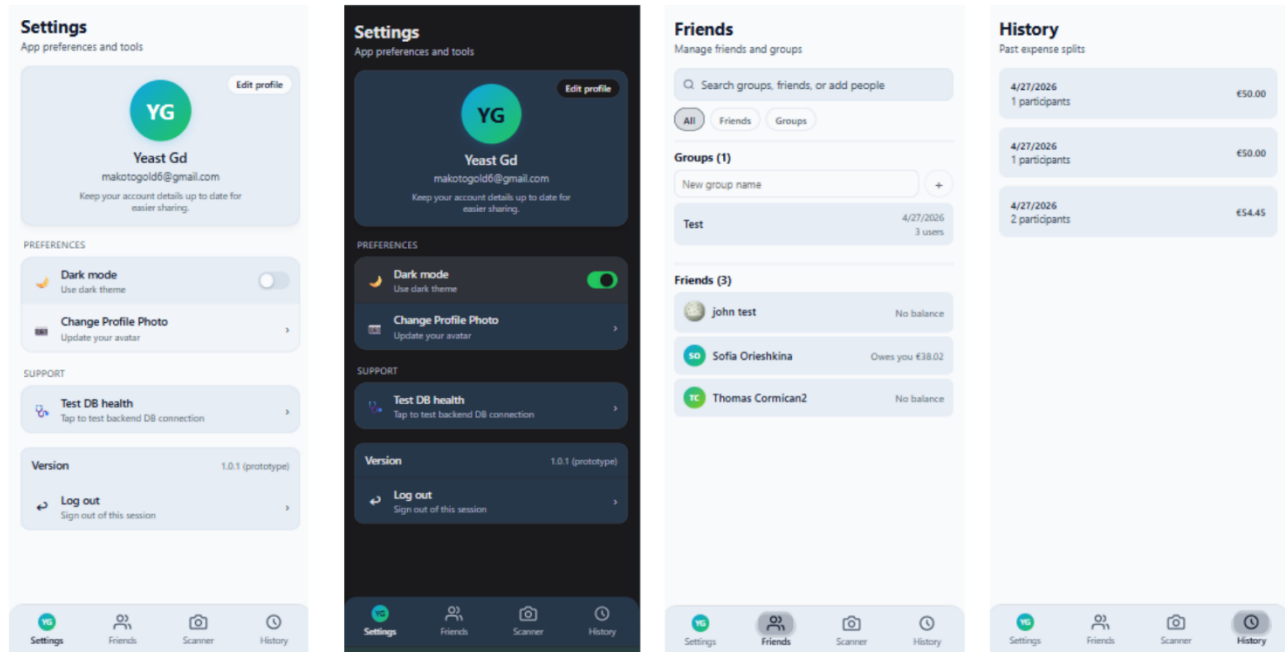
4. Architecture diagrams

Chill Bill utilizes serverless architecture to perform its responsibilities to a user. React calls the backend functions via RESTful API requests and receives a JSON response based on the route it calls. The Azure Backend employs CRUD operations to create, read, update and delete information on the SQL database depending on what is required by the user. The Azure Backend also posts binary large objects (Blob) to Azure to store receipt files e.g. (.png, .jpeg). Lastly, the Azure backend communicates with a prebuilt Azure OCR AI Document Intelligence model for receipts to enable the scanning of receipts. All responses delivered back to the user via backend are presented and displayed depending on the use case.



Part 3: User interface design





Part 4: System Tests

Scenario	Test Step	Expected Result	Actual Outcome	Result
Verify that a user can register and log in with valid details	Go to Register page and enter valid details. Submit. Then go to Login page and enter the same details.	Application should create the account and log the user in successfully.	Application creates the user account and logs the user in successfully.	Pass
Verify that login fails with incorrect password	Go to Login page. Enter valid email and incorrect password. Click Login.	Application should NOT log the user in and should show an error message.	Application fails to log the user in and displays an error message stating 'invalid credentials'.	Pass
Verify that a user can edit their profile and change their profile image	Go to settings and press 'Edit Profile'. Make changes and then select 'Save Changes'.	Application should update the user's info and store it in the database.	Application updates the user info and stores the new profile image in blob storage.	Pass

Scenario	Test Step	Expected Result	Actual Outcome	Result
Verify that the full receipt flow saves data correctly	Scan/upload receipt, add users, choose payer, choose split mode, and click Done.	Application should save event, receipt, participants, line items, and debts successfully.	All data (event, receipt image, participants, and debts) were successfully saved and persist upon reload.	Pass
Verify that percentage split calculates debts correctly	Create a receipt total and split between participants by percentage, then save.	Application should assign percentage debt amounts to non-payers.	The total was divided on a percentage basis; debts reflect the correct calculated amount.	Pass
Verify that equal split calculates debts correctly	Create a receipt total and split equally between participants, then save.	Application should assign equal debt amounts to non-payers.	The total was divided equally among all participants; debt records reflect the correct amount.	Pass
Verify that itemized split calculates debts correctly	Create receipt items, assign items to users, and save.	Application should assign debt based on item assignments only.	Debt calculations accurately reflect the specific items assigned to each individual user.	Pass
Verify that optimization reduces circular debts	Create debts in a circular pattern (A owes B, B owes C, C owes A) and open optimized repayment breakdown.	Application should reduce to fewer final repayments.	The algorithm resolved the circular debt, resulting in a simplified transaction list.	Pass
Verify that no-optimization cases are shown as already optimized	Create a case with no circular debt and open optimized repayment breakdown.	Application should show that repayments are already optimized.	The system correctly identified no circular debt and displayed the standard repayment list.	Pass
Verify that a friend can be added and removed	Search for a user, click Add Friend, then open friend details and remove friend.	Application should add and remove the friend successfully.	User was added to the friend list immediately and removed	Pass

Scenario	Test Step	Expected Result	Actual Outcome	Result
			successfully upon selecting 'Delete'.	
Verify that groups can be created, renamed and deleted	Create a group, add members, rename the group, then delete it.	Application should complete all group actions successfully.	Group was created, members added, name update persisted, and deletion removed the group from the UI.	Pass
Verify that History shows correct saved receipt details	Save a split, open History, and open the receipt details.	Application should show correct totals, participants, and item details.	The history view accurately retrieved and displayed the totals and participant breakdown.	Pass
Verify that user session persists when page is reloaded	Login and then refresh the page.	Application should keep the user logged in.	The user remains logged in after the page has been refreshed.	Pass
Verify unauthorized routes require authentication	Log out and try to open protected routes directly via URL (e.g., /friends, /history, /results).	Application should redirect to Login and block protected content.	Application redirects to the login screen when visiting unauthorized routes.	Pass